

EPIC 02 – Spring Boot Microservices

Task 1 — Create Production Structured Spring Boot Microservices

using **Java + Spring Boot**, start with these 3 services:

1. **User Service**
2. **Product Service**
3. **Order Service**

1. Create the projects

In **Spring Initializer**, create each project with:

- Project: **Maven**
- Language: **Java**
- Spring Boot: latest stable 3.x
- Java: **17**
- Packaging: **Jar**

Add these dependencies:

- Spring Web
- Spring Data JPA
- Validation
- Lombok
- MySQL Driver

2. Project structure

For example, your **Order Service** should look like this:

order-service

|

```
└─ src/main/java
|   └─ com.example.orderservice
|
|   └─ controller
|       └─ OrderController.java
|
|   └─ service
|       └─ OrderService.java
|
|   └─ repository
|       └─ OrderRepository.java
|
|   └─ entity
|       └─ Order.java
|
|   └─ dto
|       └─ OrderRequest.java
|       └─ OrderResponse.java
|
|   └─ OrderServiceApplication.java
|
└─ src/main/resources
```

└─ application.yml

Create the same type of structure for **User Service** and **Product Service**.

3. Create the Entity

Example Order.java:

```
package com.example.orderservice.entity;
```

```
import jakarta.persistence.*;
```

```
import lombok.*;
```

```
@Entity
```

```
@Table(name = "orders")
```

```
@Getter
```

```
@Setter
```

```
@NoArgsConstructor
```

```
@AllArgsConstructor
```

```
public class Order {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
```

```
private Long userId;  
  
private Long productId;  
  
private Integer quantity;  
  
}
```

4. Create Repository

```
package com.example.orderservice.repository;  
  
import com.example.orderservice.entity.Order;  
import org.springframework.data.jpa.repository.JpaRepository;  
  
public interface OrderRepository extends JpaRepository<Order, Long> {  
  
}
```

5. Create DTOs

OrderRequest.java

```
package com.example.orderservice.dto;  
  
import lombok.Data;  
  
@Data
```

```
public class OrderRequest {  
  
    private Long userId;  
    private Long productId;  
    private Integer quantity;  
}
```

OrderResponse.java

```
package com.example.orderservice.dto;
```

```
import lombok.AllArgsConstructor;
```

```
import lombok.Data;
```

```
@Data
```

```
@AllArgsConstructor
```

```
public class OrderResponse {
```

```
    private Long id;
```

```
    private Long userId;
```

```
    private Long productId;
```

```
    private Integer quantity;
```

```
}
```

6. Create Service

```
package com.example.orderservice.service;

import com.example.orderservice.dto.OrderRequest;
import com.example.orderservice.entity.Order;
import com.example.orderservice.repository.OrderRepository;
import org.springframework.stereotype.Service;

@Service
public class OrderService {

    private final OrderRepository orderRepository;

    public OrderService(OrderRepository orderRepository) {
        this.orderRepository = orderRepository;
    }

    public Order createOrder(OrderRequest request) {

        Order order = new Order();

        order.setUserId(request.getUserId());
        order.setProductId(request.getProductId());
        order.setQuantity(request.getQuantity());

        return orderRepository.save(order);
    }
}
```

```
}  
}
```

7. Create Controller

```
package com.example.orderservice.controller;
```

```
import com.example.orderservice.dto.OrderRequest;  
import com.example.orderservice.entity.Order;  
import com.example.orderservice.service.OrderService;  
import org.springframework.web.bind.annotation.*;
```

```
@RestController
```

```
@RequestMapping("/api/orders")
```

```
public class OrderController {
```

```
    private final OrderService orderService;
```

```
    public OrderController(OrderService orderService) {  
        this.orderService = orderService;  
    }
```

```
    @PostMapping
```

```
    public Order createOrder(@RequestBody OrderRequest request) {  
        return orderService.createOrder(request);  
    }
```

```
}
```

8. Final structure

Your **Order Service** should now contain:

controller



service



repository



Entity

and:

dto

entity

repository

service

controller

configuration